

ТОО «Колледж Хекслет»

ПРАКТИЧЕКАЯ РАБОТА

на тему: «Тестирование проекта CineTrack»

по модулю: «Прикладное программирование»

Специальность: 06130100 – Программное обеспечение (по видам)

Квалификация: 4S06130105 – Техник информационных систем

Выполнил: студент 3 курса
8 ТИС/23 группы
Есқали Бейбарыс Жеңісұлы

Допущен к защите «__» _____ 20__ г.
Преподаватель: _____ Турман.Н.Р
(подпись)

Защитил с оценкой:
_____ (буква) _____ (циф.экв.) _____ (балл)

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
1 ОПИСАНИЕ ПРОЕКТА.....	5
1.1. Общие сведения о проекте.....	5
1.2. Архитектура приложения.....	6
1.3. Основные страницы и функции.....	6
1.4. Стек технологий.....	8
2 РУЧНОЕ ТЕСТИРОВАНИЕ.....	10
2.1. Методология ручного тестирования.....	10
2.2. Чек-лист ручного тестирования.....	11
2.3. Тест-кейсы.....	12
2.4. Баг-репорты.....	15
3 UNIT-ТЕСТИРОВАНИЕ.....	18
3.1. Методология unit-тестирования.....	18
3.2. Настройка тестового окружения.....	18
3.3. Выбор компонентов для тестирования.....	20
3.4. Описание компонента ItemForm.vue.....	21
3.5. Описание компонента ItemCard.vue.....	21
3.6. Код и описание unit-тестов ItemForm.test.js.....	22
3.7. Код и описание unit-тестов ItemCard.test.js.....	23
3.8. Результаты запуска тестов.....	24
ЗАКЛЮЧЕНИЕ.....	26
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	27

ВВЕДЕНИЕ

Тестирование программного обеспечения является неотъемлемым этапом разработки любого приложения. Без систематической проверки работоспособности даже технически грамотно написанный продукт может содержать ошибки, которые негативно скажутся на пользовательском опыте и надёжности системы в целом. Особую роль тестирование играет во фронтенд-разработке, где поведение приложения напрямую зависит от взаимодействия компонентов, маршрутизации, управления состоянием и работы с браузерными API.

Объектом тестирования в данной практической работе является проект CineTrack — одностраничное веб-приложение (SPA), разработанное на фреймворке Vue 3 с использованием Vite. Приложение предназначено для персонального учёта фильмов: пользователь может добавлять, редактировать, удалять и архивировать записи, отмечать просмотренное, фильтровать список и просматривать статистику. Хранение данных реализовано через localStorage без серверной части.

Цель данной практической работы — провести комплексное тестирование приложения CineTrack: выявить дефекты путём ручной проверки и подтвердить корректность работы отдельных компонентов с помощью автоматизированных unit-тестов.

Для достижения поставленной цели необходимо решить следующие задачи:

- изучить функциональность приложения и составить чек-лист ручного тестирования;
- разработать тест-кейсы, охватывающие ключевые сценарии использования;
- выявить и задокументировать дефекты в виде баг-репортов;
- настроить тестовое окружение на базе Vitest и @vue/test-utils;
- написать unit-тесты для компонентов ItemForm и ItemCard;
- проанализировать результаты тестирования и сформулировать выводы о качестве проекта.

Предмет исследования — функциональность, надёжность и корректность поведения интерфейсных компонентов приложения CineTrack.

В работе применяются следующие методы тестирования:

- ручное функциональное тестирование — проверка поведения приложения путём непосредственного взаимодействия с интерфейсом по заранее составленному чек-листу и тест-кейсам;
- unit-тестирование — автоматизированная проверка изолированных компонентов с использованием фреймворка Vitest и библиотеки @vue/test-utils.

1 ОПИСАНИЕ ПРОЕКТА

1.1. Общие сведения о проекте

CineTrack — одностраничное веб-приложение (SPA), разработанное на фреймворке Vue 3 с использованием сборщика Vite. Приложение представляет собой персональный трекер фильмов: авторизованный пользователь ведёт собственный список просмотренного и запланированного к просмотру, управляет записями и отслеживает прогресс через страницу статистики. Приложение не имеет серверной части — все данные хранятся в localStorage браузера. Общие сведения о проекте представлены в таблице 1.

Таблица 1 — Общие сведения о проекте CineTrack

№	Параметр	Значение
N	1	2
1	Название проекта	CineTrack
2	Тип приложения	SPA (Single Page Application)
3	Фреймворк	Vue 3 (Composition API, script setup)
4	Сборщик	Vite 8.x
5	Маршрутизация	Vue Router 4
6	Хранилище данных	localStorage (без серверной части)
7	Внешний API	The Movie Database (TMDB)
8	Версия Node.js	^20.19.0 или >=22.12.0
9	Количество маршрутов	11
10	Количество компонентов	7
11	Количество composables	4
12	Количество сервисов	3
Примечание - составлено автором на основе проведенного исследования		

1.2. Архитектура приложения

Архитектура CineTrack построена по трёхслойному принципу: слой сервисов, слой composables и слой представления. Такое разделение обеспечивает независимость бизнес-логики от деталей реализации интерфейса.

Слой сервисов (src/services/) отвечает за взаимодействие с внешними источниками данных. Включает три модуля: storage.js — операции с localStorage; auth.js — регистрация, вход, выход, сессия; tmdb.js — поиск фильмов через TMDB API.

Слой composables (src/composables/) реализует бизнес-логику в виде singleton-модулей: useItems.js (CRUD, архив, счётчики), useAuth.js (состояние авторизации), useSettings.js (тема, схема, режим), useLoader.js (индикатор загрузки).

Слой представления (src/views/, src/components/) включает 11 страниц-маршрутов и 7 переиспользуемых компонентов. Компоненты не содержат бизнес-логики — получают данные через composables и генерируют события.

Навигация защищена guards в router/index.js: до каждого перехода вызывается showLoader(), после — hideLoader() с задержкой 400 мс. Защищённые маршруты проверяют наличие активной сессии и перенаправляют неавторизованного пользователя на /login.

1.3. Основные страницы и функции

Приложение содержит 11 маршрутов, разделённых на публичные и защищённые. Полный перечень страниц представлен в таблице 2.

Таблица 2 — Страницы приложения CineTrack

№	Маршрут	Компонент	Доступ	Назначение
<i>N</i>	<i>I</i>	<i>2</i>	<i>3</i>	<i>4</i>
1	/	LandingView	Публичный	Приветственная страница с описанием возможностей
2	/login	LoginView	Публичный	Форма входа и регистрации пользователя
3	/films	HomeView	Защищённый	Список фильмов, фильтры, счётчики
4	/create	CreateView	Защищённый	Форма добавления нового фильма

№	Маршрут	Компонент	Доступ	Назначение
<i>N</i>	<i>1</i>	<i>2</i>	<i>3</i>	<i>4</i>
5	/edit/:id	EditView	Защищённый	Форма редактирования существующего фильма
6	/film/:id	FilmView	Защищённый	Детальная страница фильма с постером
7	/archive	ArchiveView	Защищённый	Список удалённых фильмов с восстановлением
9	/stats	StatsView	Защищённый	Статистика просмотра и распределение по жанрам
10	/settings	SettingsView	Защищённый	Настройки темы, цветовой схемы и отображения
11	/about	AboutView	Защищённый	Информация о проекте
12	/:pathMatch(.*)	NotFoundView	Публичный	Страница ошибки 404
Примечание - составлено автором на основе проведенного исследования				

Основные функции приложения описаны в таблице 3.

Таблица 3 — Основные функции приложения CineTrack

№	Функция	Описание
<i>N</i>	<i>1</i>	<i>2</i>
2	Регистрация и вход	Создание учётной записи, хранение сессии в localStorage
3	Добавление фильма	Форма с полями: название, описание, жанр, постер, рейтинг, год; поиск через TMDB API
4	Редактирование	Изменение любого поля записи с сохранением в localStorage
5	Мягкое удаление	Устанавливается метка deletedAt, фильм перемещается в архив
6	Восстановление	Сброс метки deletedAt, фильм возвращается в основной список
7	Удаление навсегда	Физическое удаление записи из localStorage без возможности восстановления
8	Отметка просмотра	Переключение флага isDone; просмотренные фильмы визуально выделяются

№	Функция	Описание
<i>N</i>	<i>1</i>	<i>2</i>
9	Фильтрация	Одновременная фильтрация по поиску, жанру и статусу просмотра
10	Статистика	Счётчики, прогресс-бар и диаграмма распределения по жанрам
11	Настройки	Светлая/тёмная тема, четыре цветовые схемы, режим карточками или таблицей
Примечание - составлено автором на основе проведенного исследования		

1.4. Стек технологий

Выбор технологий обусловлен требованиями технического задания и современными стандартами фронтенд-разработки. Используемые инструменты описаны в таблице 4.

Таблица 4 — Стек технологий проекта CineTrack

№	Технология	Версия	Назначение
<i>N</i>	<i>1</i>	<i>2</i>	<i>3</i>
1	Vue 3	^3.5.32	Основной фреймворк; Composition API и script setup для модульной логики
2	Vite	^8.0.8	Сборщик; быстрый запуск в dev-режиме и оптимизированная production-сборка
3	Vue Router	^4.6.4	Клиентская маршрутизация SPA с navigation guards для защиты маршрутов
4	localStorage	—	Браузерное хранилище; инкапсулировано в storage.js с обработкой ошибок
5	TMDB API	v3	Внешний API для поиска фильмов и получения метаданных
6	Vitest	latest	Фреймворк unit-тестирования, нативно совместимый с Vite
7	@vue/test-utils	latest	Официальная библиотека для монтирования и тестирования компонентов Vue
8	jsdom	latest	Эмуляция DOM-среды браузера при запуске тестов в Node.js
Примечание - составлено автором на основе проведенного исследования			

Vue 3 и Composition API выбраны как основной инструмент: реактивность через `ref` и `computed`, компактный синтаксис `script setup` и `composables` для переиспользуемой бизнес-логики.

`localStorage` выбран как встроенный браузерный механизм без серверной инфраструктуры. Все обращения инкапсулированы в `services/storage.js` через `try/catch`.

2 РУЧНОЕ ТЕСТИРОВАНИЕ

2.1. Методология ручного тестирования

Ручное тестирование представляет собой метод проверки программного обеспечения, при котором тестировщик взаимодействует с приложением непосредственно через пользовательский интерфейс, имитируя действия реального пользователя. В отличие от автоматизированного тестирования, ручная проверка позволяет оценить не только корректность работы функций, но и удобство интерфейса, логичность навигации и визуальное соответствие ожиданиям пользователя.

Параметры тестовой среды представлены в таблице 5.

Таблица 5 — Тестовая среда

№	Параметр	Значение
1	1	2
2	Браузер	Google Chrome последней версии
3	Операционная система	Windows 10 / 11
4	Разрешение (десктоп)	1920×1080
5	Разрешение (мобильное)	375×667 (эмуляция в DevTools)
6	Способ запуска	npm run dev (локальный сервер Vite)
7	Инструменты	Chrome DevTools: консоль, эмуляция устройств, вкладка Application → localStorage
Примечание - составлено автором на основе проведенного исследования		

Тестирование проводилось по следующим направлениям:

- функциональное тестирование — проверка соответствия поведения заявленным требованиям: авторизация, CRUD, фильтрация, архивирование, статистика;
- тестирование граничных значений — проверка форм при пустых полях, коротких значениях и некорректных данных;
- тестирование навигации — корректность переходов, работа кнопок «Назад», перенаправления при отсутствии авторизации;
- тестирование сохранения данных — корректная запись в localStorage и восстановление после перезагрузки;
- тестирование адаптивности — корректное отображение на мобильном разрешении 375px через Chrome DevTools.

2.2. Чек-лист ручного тестирования

Чек-лист составлен на основе анализа функциональных требований приложения. Каждый пункт представляет атомарную проверку одного аспекта поведения системы (таблица 6).

Таблица 6 — Чек-лист ручного тестирования

№	Проверка	Статус
<i>N</i>	<i>I</i>	<i>2</i>
1	Проект запускается командой <code>npm run dev</code> без ошибок в терминале	Пройдено
2	Страница <code>/login</code> открывается без ошибок в консоли браузера	Пройдено
3	Форма регистрации создаёт нового пользователя при корректных данных	Пройдено
4	При повторной регистрации с существующим логином отображается сообщение об ошибке	Пройдено
5	Вход с корректными логином и паролем выполняется успешно	Пройдено
6	При вводе неверного пароля отображается сообщение об ошибке	Пройдено
7	Неавторизованный пользователь при переходе на <code>/films</code> перенаправляется на <code>/login</code>	Пройдено
8	Форма добавления не отправляется с незаполненными обязательными полями	Пройдено
9	При вводе названия короче 2 символов отображается сообщение об ошибке	Пройдено
10	После корректного заполнения формы новый фильм появляется в списке	Пройдено
11	Кнопка «Просмотрено» переключает статус, карточка визуально обновляется	Пройдено
12	Удалённый фильм исчезает с <code>/films</code> и появляется на <code>/archive</code>	Пройдено
13	Восстановленный из архива фильм возвращается в основной список	Пройдено
14	Фильм, удалённый навсегда, полностью исчезает из приложения	Пройдено
15	Поиск по названию фильтрует список в реальном времени без перезагрузки	Пройдено
16	Фильтр по жанру отображает только фильмы выбранного жанра	Пройдено
17	Данные сохраняются в <code>localStorage</code> и восстанавливаются после перезагрузки	Пройдено

№	Проверка	Статус
<i>N</i>	<i>I</i>	<i>2</i>
18	Страница /stats отображает корректные счётчики и прогресс-бар	Пройдено
19	Переключение тёмной/светлой темы применяется немедленно и сохраняется	Пройдено
20	При переходе на несуществующий URL отображается страница 404 с кнопкой возврата	Пройдено
Примечание - составлено автором на основе проведенного исследования		

2.3. Тест-кейсы

Тест-кейсы разработаны для проверки ключевых сценариев использования приложения CineTrack (таблица 7).

Таблица 7 — Тест-кейсы приложения CineTrack

№	ID	Название теста	Шаги	Ожидаемый результат	Фактический результат	Статус
<i>N</i>	<i>1</i>	<i>2</i>	<i>3</i>	<i>4</i>	<i>5</i>	<i>6</i>
1	ТС-01	Проверка запуска проекта	1. Открыть терминал в корневой папке 2. Выполнить <code>npm run dev</code> 3. Открыть <code>localhost:5173</code>	Проект запускается без ошибок, открывается лендинг-страница CineTrack	Проект запустился, лендинг отображается корректно	Пройдено
2	ТС-02	Регистрация нового пользователя	1. Перейти на <code>/login</code> 2. Нажать «Регистрация» 3. Ввести логин (>3 символов) 4. Ввести пароль (>4 символов) 5. Нажать «Зарегистрироваться»	Учётная запись создана, переход на главную, отображается имя пользователя	Регистрация выполнена, выполнен переход	Пройдено
3	ТС-03	Повторная регистрация	1. Перейти на <code>/login</code>	Регистрация не выполняется,	Появилось сообщение об ошибке	Пройдено

№	ID	Название теста	Шаги	Ожидаемый результат	Фактический результат	Статус
N	I	2	3	4	5	6
		ия с существующим логином	2. Открыть «Регистрация» 3. Ввести уже существующий логин 4. Нажать «Зарегистрироваться»	отображается сообщение об ошибке		
4	ТС-04	Вход с корректными данными	1. Перейти на /login 2. Ввести корректный логин 3. Ввести корректный пароль 4. Нажать «Войти»	Вход выполнен, пользователь перенаправляется на /films	Вход выполнен, список фильмов отображается	Пройдено
5	ТС-05	Вход с неверным паролем	1. Перейти на /login 2. Ввести корректный логин 3. Ввести неверный пароль 4. Нажать «Войти»	Вход не выполняется, отображается сообщение «Неверный пароль»	Вход не выполнен, сообщение об ошибке появилось	Пройдено
6	ТС-06	Добавление нового фильма	1. Перейти на /create 2. Ввести название «Интерстеллар» 3. Ввести описание (>10 символов) 4. Выбрать жанр «Фантастика» 5. Нажать «Добавить в список»	Фильм добавляется в список, выполняется переход, запись видна	Фильм добавлен, переход выполнен	Пройдено
7	ТС-07	Отправка формы с	1. Перейти на /create	Форма не отправляется,	Форма не отправлена, все	Пройдено

№	ID	Название теста	Шаги	Ожидаемый результат	Фактический результат	Статус
N	I	2	3	4	5	6
		пустыми полями	2. Не заполнять ни одно поле 3. Нажать «Добавить в список»	отображаются три сообщения об ошибках валидации	три ошибки отображаются	
8	ТС-08	Отметка фильма как просмотренного	1. Перейти на /films 2. Выбрать фильм со статусом «Не просмотрено» 3. Нажать кнопку «✓» на карточке	Статус меняется, карточка визуально обновляется, счётчик увеличивается на 1	Статус изменён, счётчик обновился	Пройдено
9	ТС-09	Мягкое удаление и восстановление	1. На /films нажать «X» на карточке 2. Подтвердить удаление 3. Перейти на /archive 4. Нажать «Восстановить»	Фильм исчезает с /films, появляется на /archive; после восстановления возвращается	Фильм перемещён в архив и успешно восстановлен	Пройдено
10	ТС-10	Сохранение данных после перезагрузки	1. Авторизоваться 2. Добавить фильм 3. Отметить фильм просмотренным 4. Закрыть вкладку 5. Открыть приложение повторно	Данные сохранены: фильм в списке, статус просмотра сохранён, сессия активна	Данные сохранены, изменения не потеряны, сессия активна	Пройдено
Примечание - составлено автором на основе проведенного исследования						

2.4. Баг-репорты

В ходе ручного тестирования приложения CineTrack были выявлены три дефекта. Каждый дефект оформлен в виде баг-репорта с указанием шагов воспроизведения, ожидаемого и фактического результата.

Таблица 8 — Баг-репорт BUG-01

№	Поле	Содержание
<i>N</i>	<i>I</i>	<i>2</i>
1	ID	BUG-01
2	Название	Постер и метаданные TMDb не сохраняются при добавлении фильма
3	Компонент	src/composables/useItems.js — функция addItem()
4	Приоритет	Средний
5	Шаги воспроизведения	1. Авторизоваться и перейти на /create 2. Ввести название фильма в поле поиска 3. Выбрать фильм из списка TMDb (поля заполняются автоматически) 4. Нажать «Добавить в список» 5. Перейти на страницу фильма (/film/:id)
6	Ожидаемый результат	На странице фильма отображается постер из TMDb, заполнены поля год и рейтинг
7	Фактический результат	Постер не отображается, поля год и рейтинг пусты — данные TMDb не сохраняются
8	Причина	В функции addItem() объект newItem содержит только базовые поля; posterPath, rating, year, tmdbId из data игнорируются
9	Предложение по исправлению	Добавить поля posterPath, rating, year, tmdbId в объект newItem внутри addItem() в файле useItems.js
Примечание - составлено автором на основе проведенного исследования		

Таблица 9 — Баг-репорт BUG-02

№	Поле	Содержание
<i>N</i>	<i>I</i>	<i>2</i>
1	ID	BUG-02
2	Название	После добавления и редактирования перенаправление на «/» вместо «/films»
3	Компонент	src/views/CreateView.vue, src/views/EditView.vue
4	Приоритет	Средний
5	Шаги воспроизведения	1. Авторизоваться и перейти на /create 2. Заполнить форму корректными данными 3. Нажать «Добавить в список» 4. Наблюдать, на какую страницу выполняется переход
6	Ожидаемый результат	Пользователь перенаправляется на /films, где виден добавленный фильм
7	Фактический результат	Пользователь перенаправляется на «/» (лендинг), новый фильм не виден без ручного перехода
8	Причина	В handleSubmit() файлов CreateView.vue и EditView.vue указан router.push('/'), тогда как список фильмов на /films
9	Предложение по исправлению	Заменить router.push('/') на router.push('/films') в handleSubmit() в CreateView.vue и EditView.vue
Примечание - составлено автором на основе проведенного исследования		

Таблица 10 — Баг-репорт BUG-03

№	Поле	Содержание
<i>N</i>	<i>I</i>	<i>2</i>
1	ID	BUG-03
2	Название	Непоследовательная навигация: «← Назад» и «Отмена» ведут на разные страницы
3	Компонент	src/views/CreateView.vue, src/views/EditView.vue, src/components/ItemForm.vue
4	Приоритет	Низкий
5	Шаги воспроизведения	1. Авторизоваться, перейти на /films 2. Нажать «+ Добавить фильм» → открывается /create 3. Нажать ссылку «← Назад» в шапке страницы

№	Поле	Содержание
<i>N</i>	<i>I</i>	<i>2</i>
		4. Вернуться на /create, нажать кнопку «Отмена» в форме
6	Ожидаемый результат	Оба элемента навигации ведут на одну страницу — /films
7	Фактический результат	«← Назад» ведёт на «/» (лендинг), «Отмена» ведёт на /films — поведение на одной странице несогласовано
8	Причина	В шаблоне CreateView.vue ссылка «← Назад» содержит to="/", тогда как в ItemForm.vue кнопка «Отмена» содержит to="/films"
9	Предложение по исправлению	В CreateView.vue и EditView.vue заменить to="/" на to="/films" у ссылки «← Назад»
Примечание - составлено автором на основе проведенного исследования		

3 UNIT-ТЕСТИРОВАНИЕ

3.1. Методология unit-тестирования

Unit-тестирование — метод автоматизированного тестирования, при котором проверяется работа наименьшей изолированной единицы кода: функции, метода или компонента. Unit-тесты запускаются автоматически, выполняются за секунды и дают воспроизводимый результат при каждом запуске.

В данной работе unit-тестирование проводилось по следующим принципам: изоляция — каждый тест независим и не зависит от внешних сервисов; читаемость — название теста описывает проверяемое поведение; минимальность — каждый тест проверяет ровно один сценарий; воспроизводимость — результат одинаков при каждом запуске.

Для каждого компонента написаны три теста, покрывающие три обязательных сценария (таблица 11).

Таблица 11 — Сценарии unit-тестирования

№	Сценарий	Что проверяется
<i>N</i>	<i>I</i>	<i>2</i>
1	Компонент отображается	Наличие ключевых элементов интерфейса после монтирования: поля ввода, кнопки, текст
2	Пользовательское действие работает	Реакция компонента на клик или ввод: генерация события, изменение состояния
3	Валидация и обработка ошибок работает	Появление сообщений об ошибках при некорректном или пустом вводе
Примечание - составлено автором на основе проведенного исследования		

3.2. Настройка тестового окружения

Для unit-тестирования Vue 3-компонентов использован следующий набор инструментов: Vitest — фреймворк для тестирования, разработанный для проектов на Vite; @vue/test-utils — официальная библиотека для монтирования компонентов; jsdom — эмуляция DOM-среды браузера в Node.js.

Установка зависимостей. Выполнить команду в корневой папке проекта:

```
npm install -D vitest @vue/test-utils jsdom
```

Обновление package.json. Добавить команду запуска тестов в секцию scripts:

```
"scripts": {
  "dev": "vite",
  "build": "vite build",
  "preview": "vite preview",
  "test": "vitest"
}
```

Обновление vite.config.js. Добавить блок test с указанием тестовой среды:

```
import { defineConfig } from 'vite'
import vue from '@vitejs/plugin-vue'

export default defineConfig({
  plugins: [vue()],
  test: {
    environment: 'jsdom',
    globals: true,
  },
})
```

Параметр environment: 'jsdom' задаёт эмулятор браузерного DOM. Параметр globals: true делает describe, it, expect доступными без явного импорта. Структура тестовых файлов представлена в таблице 12.

Таблица 12 — Структура тестовых файлов

№	Файл	Расположение	Назначение
<i>N</i>	<i>I</i>	<i>2</i>	<i>3</i>
1	vite.config.js	корень проекта	Конфигурация Vite с добавленным блоком test
2	ItemForm.test.js	src/components/	Unit-тесты компонента ItemForm.vue (3 теста)
3	ItemCard.test.js	src/components/	Unit-тесты компонента ItemCard.vue (3 теста)
Примечание - составлено автором на основе проведенного исследования			

Принцип работы заглушек. Компонент ItemForm.vue содержит дочерний компонент TmdbSearch.vue, который выполняет реальные HTTP-запросы. В тестах он заменяется пустым шаблоном. Аналогично router-link заменяется встроенной заглушкой RouterLinkStub из @vue/test-utils, так как маршрутизатор не инициализирован в изолированной среде:

```

mount(ItemForm, {
  global: {
    stubs: {
      TmdbSearch: { template: '<div />' },
      RouterLink: RouterLinkStub,
    },
  },
})

```

3.3. Выбор компонентов для тестирования

Для unit-тестирования выбраны компоненты ItemForm.vue и ItemCard.vue. Выбор обусловлен тем, что эти компоненты охватывают все три обязательных сценария тестирования (таблица 13).

Таблица 13 — Обоснование выбора компонентов

№	Компонент	Назначение	Покрываемые сценарии
<i>N</i>	<i>1</i>	<i>2</i>	<i>3</i>
1	ItemForm.vue	Универсальная форма добавления и редактирования фильма; содержит поля ввода, валидацию, кнопку отправки	Рендеринг полей и кнопки; валидация пустой формы; генерация события submit с данными
2	ItemCard.vue	Карточка фильма в списке; отображает название, жанр, описание и кнопки действий	Рендеринг названия и жанра; событие toggle при клике; событие delete при клике
Примечание - составлено автором на основе проведенного исследования			

3.4. Описание компонента ItemForm.vue

Компонент `ItemForm.vue` расположен в папке `src/components/` и используется на страницах `/create` (создание) и `/edit/:id` (редактирование). Универсальность достигается через входные параметры — пропсы.

Структура шаблона. Компонент состоит из: блока поиска через TMDb (дочерний компонент `TmdbSearch`); блока предпросмотра постера (если `posterPath` заполнен); поля «Название» (`input`, `id="f-title"`); поля «Описание» (`textarea`, `id="f-desc"`); поля «Жанр» (`select`, `id="f-cat"`, 11 жанров); кнопок «Отмена» и «Отправить».

Пропсы и события. Принимает: `initial` (объект с полями `title`, `description`, `category`, `posterPath`, `rating`, `year`, `tmdbId`; по умолчанию пустые) и `submitLabel` (текст кнопки, по умолчанию «Сохранить»). Генерирует событие `submit` с объектом данных формы.

Логика валидации. Функция `validate()` проверяет три условия: название — не пустое и минимум 2 символа; описание — не пустое и минимум 10 символов; жанр — выбрано из списка. При нарушении записывает сообщение в объект `errors`. Если все проверки пройдены — генерирует событие `submit`.

3.5. Описание компонента `ItemCard.vue`

Компонент `ItemCard.vue` расположен в папке `src/components/` и отображает одну карточку фильма на странице `/films`. Компонент «немой» (`dumb component`): не содержит бизнес-логики, все данные получает через пропсы, все действия передаёт через события.

Структура шаблона. Карточка состоит из двух секций. Первая — постер (ссылка-картинка или заглушка ). Вторая — тело: заголовок (жанр, бейдж «Просмотрено», рейтинг), название-ссылка на `/film/:id`, описание, подвал с датой добавления и тремя кнопками действий.

Кнопки действий. Три элемента: кнопка «✓»/«↔» генерирует событие `toggle` с `item.id`; ссылка «👉» ведёт на `/edit/:id`; кнопка «✕» генерирует событие `delete` с `item.id`. Кнопки используют модификатор `@click.stop` для предотвращения всплытия.

Пропсы и события. Принимает один обязательный пропс `item` (поля: `id`, `title`, `description`, `category`, `createdAt`, `isDone`, `deletedAt`, `posterPath`, `rating`, `year`). Генерирует события `toggle` и `delete` с аргументом `item.id`.

Визуальные состояния. Обычное и «просмотрено» (класс `card--done`: полупрозрачность, зачёркивание названия). Переход реактивный при изменении флага `isDone`.

3.6. Код и описание unit-тестов ItemForm.test.js

Файл ItemForm.test.js содержит три теста в группе describe('ItemForm'). В начале объявляется объект заглушек globalStubs, передаваемый при монтировании каждого теста:

```
import { describe, it, expect } from 'vitest'
import { mount, RouterLinkStub } from '@vue/test-utils'
import ItemForm from './ItemForm.vue'

const globalStubs = {
  stubs: {
    TmdbSearch: { template: '<div />' },
    RouterLink: RouterLinkStub,
  },
}
```

Тест 1 — отображение полей ввода и кнопки. Проверяет наличие в DOM элементов #f-title, #f-desc, #f-cat и кнопки с корректным текстом после монтирования компонента.

```
it('отображает поля ввода и кнопку отправки', () => {
  const wrapper = mount(ItemForm, {
    props: { submitLabel: 'Добавить в список' },
    global: globalStubs,
  })
  expect(wrapper.find('#f-title').exists()).toBe(true)
  expect(wrapper.find('#f-desc').exists()).toBe(true)
  expect(wrapper.find('#f-cat').exists()).toBe(true)
  expect(wrapper.find('button[type="submit"]').text())
    .toContain('Добавить в список')
})
```

Тест 2 — ошибки валидации при пустой форме. Имитирует отправку пустой формы через trigger('submit'). Проверяет наличие трёх сообщений об ошибках в тексте компонента. Ключевое слово async/await необходимо, так как обновление DOM в Vue асинхронно.

```
it('показывает ошибки при отправке пустой формы', async ()
=> {
  const wrapper = mount(ItemForm, { global: globalStubs })
  await wrapper.find('form').trigger('submit')
  expect(wrapper.text()).toContain('Введите название')
  expect(wrapper.text()).toContain('Введите описание')
  expect(wrapper.text()).toContain('Выберите жанр')
})
```

Тест 3 — событие submit с корректными данными. Заполняет поля через setValue(), отправляет форму, проверяет что событие submit сгенерировано и содержит правильные данные.

```
it('генерирует событие submit с корректными данными', async
() => {
  const wrapper = mount(ItemForm, {
    props: { submitLabel: 'Сохранить' },
    global: globalStubs,
  })
  await wrapper.find('#f-title').setValue('Интерстеллар')
  await wrapper.find('#f-desc').setValue(
    'Фильм о путешествии через космическую червоточину'
  )
  await wrapper.find('#f-cat').setValue('Фантастика')
  await wrapper.find('form').trigger('submit')
  const emitted = wrapper.emitted('submit')
  expect(emitted).toBeTruthy()
  expect(emitted[0][0].title).toBe('Интерстеллар')
  expect(emitted[0][0].category).toBe('Фантастика')
})
```

3.7. Код и описание unit-тестов ItemCard.test.js

Файл ItemCard.test.js содержит три теста в группе describe('ItemCard'). В начале объявляется тестовый объект testItem и вспомогательная функция mountCard:

```
import { describe, it, expect } from 'vitest'
import { mount, RouterLinkStub } from '@vue/test-utils'
import ItemCard from './ItemCard.vue'

const testItem = {
  id: 'abc-123', title: 'Тёмный рыцарь',
  description: 'Бэтмен против Джокера в Готэм-сити',
  category: 'Боевик', createdAt: Date.now(),
  isDone: false, deletedAt: null,
  posterPath: null, rating: 9.0, year: '2008',
}
const mountCard = (item = testItem) =>
  mount(ItemCard, {
    props: { item },
    global: { stubs: { RouterLink: RouterLinkStub } },
  })
```

Тест 4 — отображение названия и жанра. Проверяет наличие строк «Тёмный рыцарь» и «Боевик» в текстовом контенте компонента через `wrapper.text()`. Тест синхронный — данные передаются статически через пропс.

```
it('отображает название и жанр фильма', () => {
  const wrapper = mountCard()
  expect(wrapper.text()).toContain('Тёмный рыцарь')
  expect(wrapper.text()).toContain('Боевик')
})
```

Тест 5 — событие `toggle` при нажатии кнопки просмотра. Находит все кнопки через `findAll('button')`: `router-link` рендерится как `<a>` и в выборку не попадает. Первая кнопка — кнопка отметки просмотра. Проверяет событие `toggle` с правильным аргументом.

```
it('отправляет toggle при нажатии кнопки просмотра', async () => {
  const wrapper = mountCard()
  const buttons = wrapper.findAll('button')
  await buttons[0].trigger('click')
  expect(wrapper.emitted('toggle')).toBeTruthy()
  expect(wrapper.emitted('toggle')[0][0]).toBe('abc-123')
})
```

Тест 6 — событие `delete` при нажатии кнопки удаления. Находит кнопку удаления по уникальному CSS-классу `.action-btn--danger`. Проверяет событие `delete` с правильным аргументом. Модификатор `@click.stop` останавливает всплытие, но не препятствует генерации события.

```
it('отправляет delete при нажатии кнопки удаления', async () => {
  const wrapper = mountCard()
  await wrapper.find('.action-btn--danger').trigger('click')
  expect(wrapper.emitted('delete')).toBeTruthy()
  expect(wrapper.emitted('delete')[0][0]).toBe('abc-123')
})
```

3.8. Результаты запуска тестов

После размещения тестовых файлов в `src/components/` выполняется запуск командой `npm test`. Vitest автоматически обнаруживает файлы `.test.js`, монтирует компоненты в среде `jsdom` и выполняет проверки. Общее время выполнения шести тестов не превышает двух секунд. Результаты приведены в таблице 14.

Таблица 14 — Результаты выполнения unit-тестов

№	Тестовый файл	Название теста	Результат
<i>N</i>	<i>I</i>	<i>2</i>	<i>3</i>
1	ItemForm.test.js	отображает поля ввода и кнопку отправки	Пройдено
2	ItemForm.test.js	показывает ошибки при отправке пустой формы	Пройдено
3	ItemForm.test.js	генерирует событие submit с корректными данными	Пройдено
4	ItemCard.test.js	отображает название и жанр фильма	Пройдено
5	ItemCard.test.js	отправляет toggle при нажатии кнопки просмотра	Пройдено
6	ItemCard.test.js	отправляет delete при нажатии кнопки удаления	Пройдено
Примечание - составлено автором на основе проведенного исследования			

Все шесть тестов завершились успешно. Покрытие включает оба компонента и все три обязательных сценария: рендеринг, пользовательское действие и валидацию. (рисунок 1)

```
sanskrit@aspire-archlinux:~/Projects/Cinetrack$ npm test

> cinetrack@0.0.0 test
> vitest

DEV v4.1.8 /home/sanskrit/Projects/Cinetrack

✓ src/components/ItemCard.test.js (3 tests) 80ms
✓ src/components/ItemForm.test.js (3 tests) 73ms

Test Files  2 passed (2)
Tests       6 passed (6)
Start at    02:49:07
Duration    996ms (transform 267ms, setup 0ms, import 417ms, tests 153ms, environment 1.13s)

PASS Waiting for file changes...
press h to show help, press q to quit
```

Рисунок 1 – Результаты тестов

Примечание - составлено автором на основе проведенного исследования

ЗАКЛЮЧЕНИЕ

В ходе выполнения данной практической работы было проведено комплексное тестирование веб-приложения CineTrack, разработанного на фреймворке Vue 3. Работа включала два этапа: ручное тестирование и автоматизированное unit-тестирование.

На этапе ручного тестирования составлен чек-лист из 20 пунктов, охватывающий авторизацию, управление фильмами, фильтрацию, архивирование, статистику, настройки и навигацию. Все 20 пунктов получили статус «Пройдено», что свидетельствует об общей работоспособности приложения. Разработанные 10 тест-кейсов охватили наиболее критичные пользовательские сценарии — все успешно пройдены.

В ходе тестирования выявлены три дефекта. BUG-01 — функциональный: данные постера, рейтинга и года TMDb не сохраняются при добавлении фильма (addItem игнорирует эти поля). BUG-02 — навигационный: после создания или редактирования пользователь перенаправляется на лендинг вместо списка фильмов. BUG-03 — несогласованность: кнопки «← Назад» и «Отмена» на одной странице ведут на разные маршруты. Все три дефекта имеют конкретные предложения по исправлению.

На этапе unit-тестирования настроено окружение на базе Vitest, @vue/test-utils и jsdom. Написаны шесть unit-тестов для компонентов ItemForm.vue и ItemCard.vue. Тесты проверяют корректность рендеринга, реакцию на действия пользователя и работу валидации. Все шесть тестов успешно пройдены.

Приложение CineTrack в целом соответствует заявленным функциональным требованиям. Выявленные дефекты носят локальный характер и устраняются без изменения архитектуры. Проведённое тестирование подтвердило ценность обоих методов: ручное позволило выявить дефекты навигации и хранения, unit-тесты обеспечили автоматизированную проверку логики компонентов.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. You E. Vue 3. Официальная документация [Электронный ресурс] / E. You. — 2024. — Режим доступа: <https://vuejs.org/guide/introduction> — Дата обращения: 20.04.2026.
2. Vue Router 4. Официальная документация [Электронный ресурс]. — 2024. — Режим доступа: <https://router.vuejs.org/guide> — Дата обращения: 20.04.2026.
3. Vite. Официальная документация [Электронный ресурс]. — 2024. — Режим доступа: <https://vitejs.dev/guide> — Дата обращения: 20.04.2026.
4. Vitest. Официальная документация [Электронный ресурс]. — 2024. — Режим доступа: <https://vitest.dev/guide> — Дата обращения: 20.04.2026.
5. Vue Test Utils. Официальная документация [Электронный ресурс]. — 2024. — Режим доступа: <https://test-utils.vuejs.org/guide> — Дата обращения: 20.04.2026.
6. The Movie Database API. Официальная документация [Электронный ресурс]. — 2024. — Режим доступа: <https://developer.themoviedb.org/docs> — Дата обращения: 21.04.2026.
7. Web Storage API [Электронный ресурс] // MDN Web Docs / Mozilla Foundation. — 2024. — Режим доступа: https://developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API — Дата обращения: 21.04.2026.
8. CSS Custom Properties (переменные) [Электронный ресурс] // MDN Web Docs / Mozilla Foundation. — 2024. — Режим доступа: https://developer.mozilla.org/en-US/docs/Web/CSS/Using_CSS_custom_properties — Дата обращения: 21.04.2026.
9. Crypto: randomUUID() [Электронный ресурс] // MDN Web Docs / Mozilla Foundation. — 2024. — Режим доступа: <https://developer.mozilla.org/en-US/docs/Web/API/Crypto/randomUUID> — Дата обращения: 21.04.2026.
10. Флэнаган, Д. JavaScript: подробное руководство / Д. Флэнаган. — 7-е изд. — СПб. : БХВ-Петербург, 2021. — 704 с. — ISBN 978-5-9775-6741-9.
11. Симпсон, К. Вы не знаете JS: ES6 и не только / К. Симпсон. — СПб. : Питер, 2017. — 333 с. — ISBN 978-5-496-02473-6.
12. Marquardt, J. Vue.js 3 Design Patterns and Best Practices / J. Marquardt. — Birmingham : Packt Publishing, 2023. — 296 p. — ISBN 978-1-80324-845-1.